Семинар #1: Компиляция и линковка

В данном семинаре детально рассматриваются процессы компиляции и линковки, а также создание и использование статических и динамических библиотек. В отличие от других семинаров, процессы компиляции и линковки могут существенно различаться в зависимости от операционной системы. Кроме того, компиляция и линковка на языке C++ имеют свои особенности по сравнению с языком C. В этом семинаре сначала будут рассмотрены основные этапы компиляции и линковки в операционной системе Linux на языке C. Затем будут подробно разобраны особенности этих процессов в языке C++. В завершение будут освещены ключевые отличия и специфика компиляции и линковки в операционной системе Windows.

Часть 1: Этапы компиляции

Компиляция – то процесс преобразования исходного кода, в машинный код, который может быть выполнен компьютером. Компилятор – это программа, которая осуществляет компиляцию.
В этом курсе мы использовали компилятор `gcc` для кода на языке C и компилятор `g++` для кода на C++.

Компиляция состоит из четырех основных этапов: препроцессинг, компиляция, ассемблирование и линковка. Если использовать компилятор `gcc`, просто передав ему имя файла командой:

```
$ gcc main.c
```

то все этапы будут выполнены автоматически, и на выходе получится готовый исполняемый файл. Однако при необходимости можно выполнить только отдельные этапы компиляции, чтобы более детально контролировать процесс. Например, можно остановиться после препроцессинга или после генерации объектного файла.

1. **Препроцессинг**
   - На этом этапе выполняется обработка директив препроцессора (`#include`, `#define`, `#ifdef` и т.д.)
   - Происходит включение заголовочных файлов, раскрытие макросов и удаление комментариев.
   - Выполнить только этот этап компиляции с помощью `gcc` можно следующим образом:

     ```
     $ gcc -E main.c -o main.i
     ```

2. **Компиляция** (не путать с компиляцией, состоящей из четырёх этапов)
   - Преобразованный на этапе препроцессинга код компилируется в ассемблерный код.
   - Результатом этого этапа является является файл с расширением `.s`.
   - Выполнить процесс компиляции до этого этапа включительно можно следующей командой:

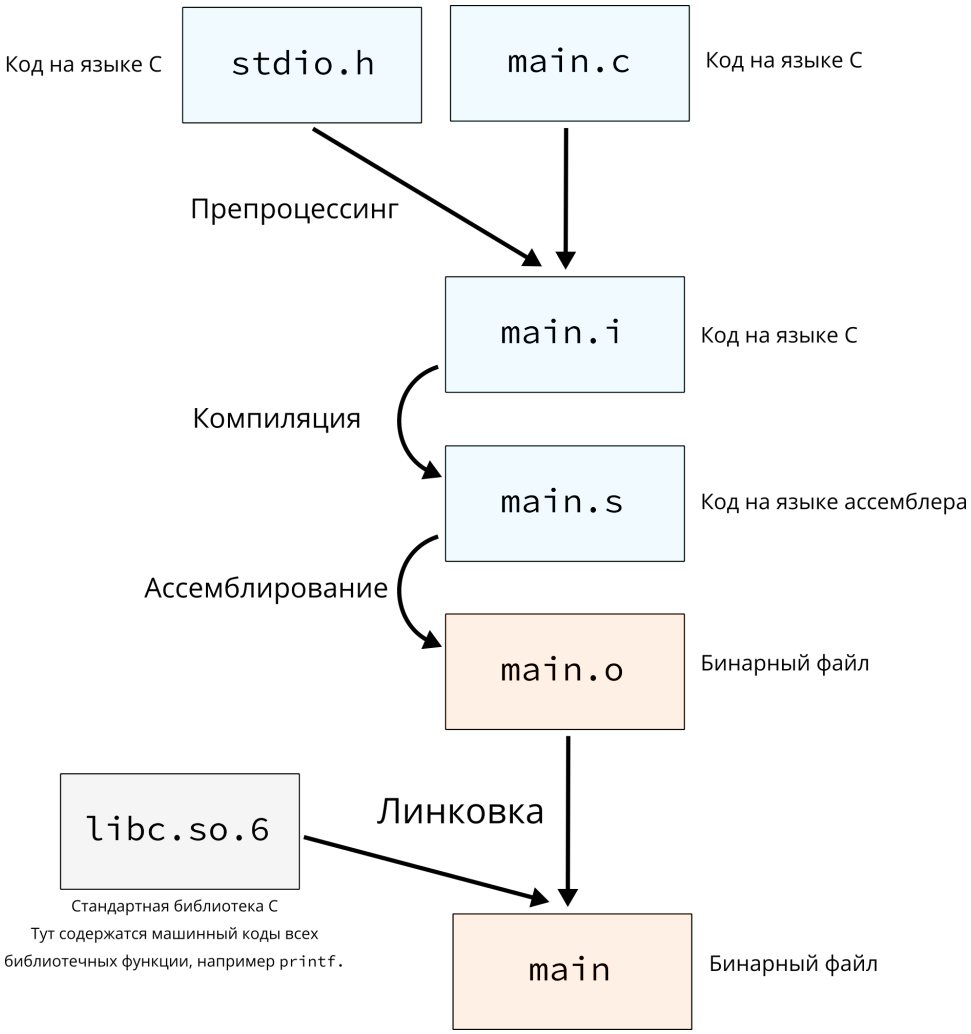
     ```
     $ gcc -S -masm=intel main.c
     ```

3. **Ассемблирования**
   - Ассемблерный код преобразуется в объектный код (машинный код), который представляет собой набор инструкций для процессора.
   - Результатом этого этапа является объектный файл с расширением `.o`.
   - Выполнить процесс компиляции до этого этапа включительно можно следующей командой:

     ```
     $ gcc -c main.c
     ```

4. **Линковка**
   - На этом этапе все объектные файлы и библиотеки связываются вместе.
   - Результатом этого этапа является исполняемый файл (на Linux исполняемые файлы обычно не имеют расширения, на Windows исполняемые файлы имеют расширение `.exe`).

Предположим, что наш проект состоит из одного файла `main.c` и из библиотек использует только стандартную библиотеку C и только функции из `stdio.h` (например, `printf`). Тогда процесс компиляции такого проекта можно показать на следующей схеме:



- На этапе препроцессинга компилятор видит директиву `#include <stdio.h>`. Поэтому он приступает к поиску заголовочного файла `stdio.h` в стандартных путях. Стандартные пути, в которых нужно искать заголовочные файлы хранятся внутри компилятора, их можно увидеть с помощью команды:

  ```
  $ echo | gcc -E -v -
  ```

- После того как он найдёт файл, он за место строки `#include <stdio.h>` вставит содержимое этого файла.
- Файл `stdio.h` содержит код на языке C и внутри этого файла также могут быть директивы `#include`, `#define` и другие. Компилятор будет выполнять все директивы пока их не останется.
- Внутри файла `stdio.h` НЕ содержатся коды стандартных функций. Там содержатся объявления (прототипы) функций. Это нужно, чтобы наша программа знала, что существуют те или иные функции.
- Код стандартных функций в уже скомпилированном виде (так называемый машинный код) содержится в библиотеке `libc.so.6` и подключается на этапе линковки.
- Различают статическую и динамическую линковку. При статической линковке машинный код всех используемых функций вставляется в окончательный исполняемый файл. При динамической линковке необходимые функции находятся в библиотеках во время выполнения.

Часть 2: Разделение кода программы на части с помощью `#include`

Простейшей способ разделение кода программы на несколько файлов – это использование директивы `#include`.

main.c

```c
#include <stdio.h>

int cube(int x)
{
    return x * x * x;
}

int main()
{
    printf("%i\n", cube(5));
}
```

main.c

```c
#include <stdio.h>
#include "cube.h"

int main()
{
    printf("%i\n", cube(5));
}
```

cube.h

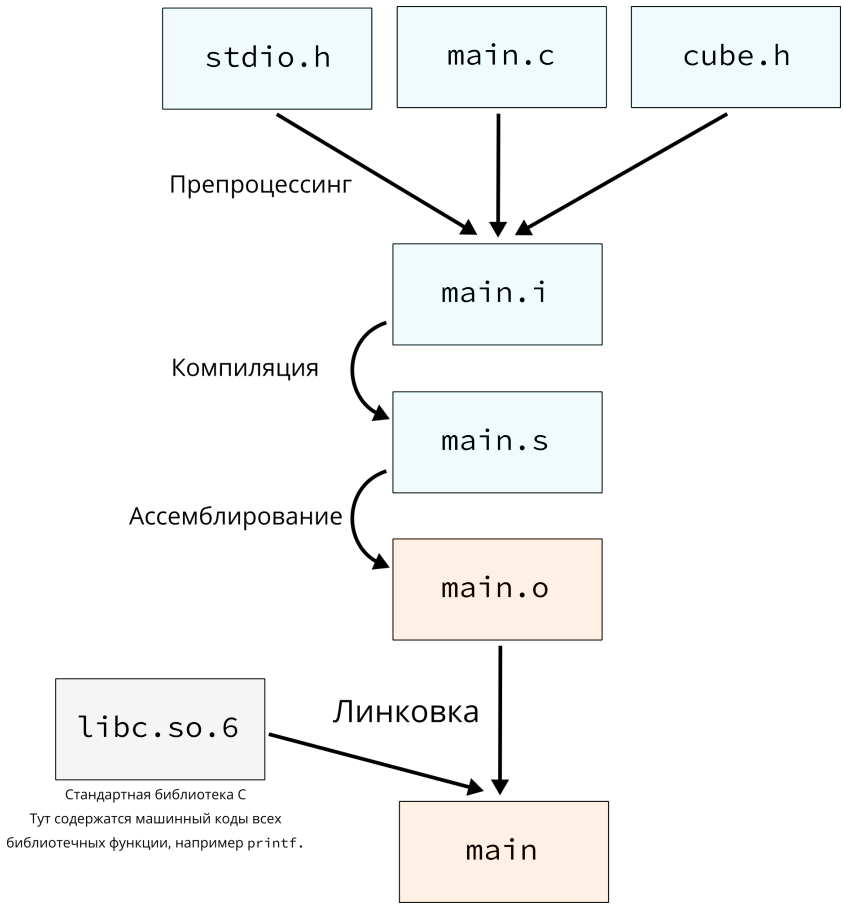
```c
#pragma once

int cube(int x)
{
    return x * x * x;
}
```

Файл `cube.h` просто вставится в файл `main.c` за место строки `#include "cube.h"` на этапе препроцессинга. При использовании угловых `<>` скобок в `#include` компилятор будет искать файл в стандартных путях, а при использовании кавычек `""` сначала поищет в текущей директории. Скомпилировать эту программу можно так:

```
$ gcc main.c
```

Передавать название файла `cube.h` компилятору не нужно, так как он найдёт этот файл через `#include`. Процесс компиляции этой программы почти не будет отличатся от предыдущей, что можно увидеть на схеме:



Часть 3: Раздельная компиляция

Раздельная компиляция – это подход, при котором программа разбивается на несколько отдельных исходных файлов, которые компилируются независимо друг от друга. В отличии от использования `#include` при раздельной компиляции части программы будут компилироваться отдельно и объединяться вместе только на этапе линковки.

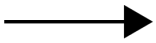
Ошибочный подход

Попробуем произвести раздельную компиляцию нашей программы, просто разделив один файл исходного кода на два: `main.c` и `cube.c` и передав на вход компилятору эти два файла.

```c
#include <stdio.h>

int cube(int x)
{
    return x * x * x;
}

int main()
{
    printf("%i\n", cube(5));
}
```



main.c

```c
#include <stdio.h>

int main()
{
    printf("%i\n", cube(5));
}
```

cube.c

```c
int cube(int x)
{
    return x * x * x;
}
```

Попробуем скомпилировать программу следующей командой:

```
$ gcc main.c cube.c
```

Произойдёт следующая ошибка:

```
main.c:5:20: error: implicit declaration of function 'cube'
```

Ошибка происходит из-за того, что компилятор пытается скомпилировать файлы `main.c` и `cube.c` раздельно, но не может скомпилировать файл `main.c` из-за того, что встечает неизвестную ему функцию `cube`. Компилятор не знает, что такая функция существует в другом файле, так как он должен скомпилировать два файла независимо друг от друга. Узнать, что в другом файле есть функция `cube` компилятор сможет только на этапе линковки.

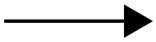
Более правильный подход

Мы можем сказать компилятору, что функция `cube` существует в другом файле программы, просто объявив (написав прототип) эту функцию в файле `main.c`.

```c
#include <stdio.h>

int cube(int x)
{
    return x * x * x;
}

int main()
{
    printf("%i\n", cube(5));
}
```



main.c

```c
#include <stdio.h>
int cube(int);

int main()
{
    printf("%i\n", cube(5));
}
```

cube.c

```c
int cube(int x)
{
    return x * x * x;
}
```

В этом случае, при компилировании файла `main.c` компилятор будет знать, что функция `cube` существует и компиляция пройдёт успешно.

Использование заголовочных файлов

Обычно, код делится на части таким образом, что в каждом файле исходного кода содержится множество функций, а не одна, как в нашем примере. Представьте, что за место файла `cube.c` с одной функцией, у нас был бы файл `mathfuncs.c`, который бы содержал 100 разных математических функций. Чтобы подключить такой файл к файлу `main.c` нам нужно было бы написать 100 прототипов функций в файле `main.c`. Если бы нам потребовалось подключить файл `mathfuncs.c` к ещё одному файлу, то нам бы потребовалось написать все 100 прототипов ещё раз. Чтобы решить эту проблему используются заголовочные файлы.

Заголовочный файл (англ. *header file*) – это текстовый файл с расширением `.h`, который содержит объявления функций, глобальных переменных, макросов, структур и других типов данных, предназначенных для использования в нескольких файлах исходного кода (`.c`). Такие файлы вставляются в файлы исходного кода с помощью `#include`.

```c
#include <stdio.h>

int cube(int x)
{
    return x * x * x;
}

int main()
{
    printf("%i\n", cube(5));
}
```

main.c

```c
#include <stdio.h>
#include "cube.h"

int main()
{
    printf("%i\n", cube(5));
}
```

cube.h

```c
#pragma once
int cube(int);
```

cube.c

```c
int cube(int x)
{
    return x * x * x;
}
```

Теперь, если даже у нас будет файл, содержащий не одну функцию, а 100 функций, то мы просто напишем соответствующий заголовочный файл, содержащий 100 прототипов функций, и будем использовать его для подключения к другим файлам.
Скомпилировать данную программу можно командой:

```
$ gcc main.c cube.c
```

Указывать файл `cube.h` не нужно, так как компилятор найдёт его и вставит, используя директиву `#include`. При использовании этой команды компилятор выполнит все этапы компилирования и получит окончательный исполняемый файл. При этом компилятор будет компилировать файлы `main.c` и `cube.c` раздельно и объединять их только на этапе линковки.

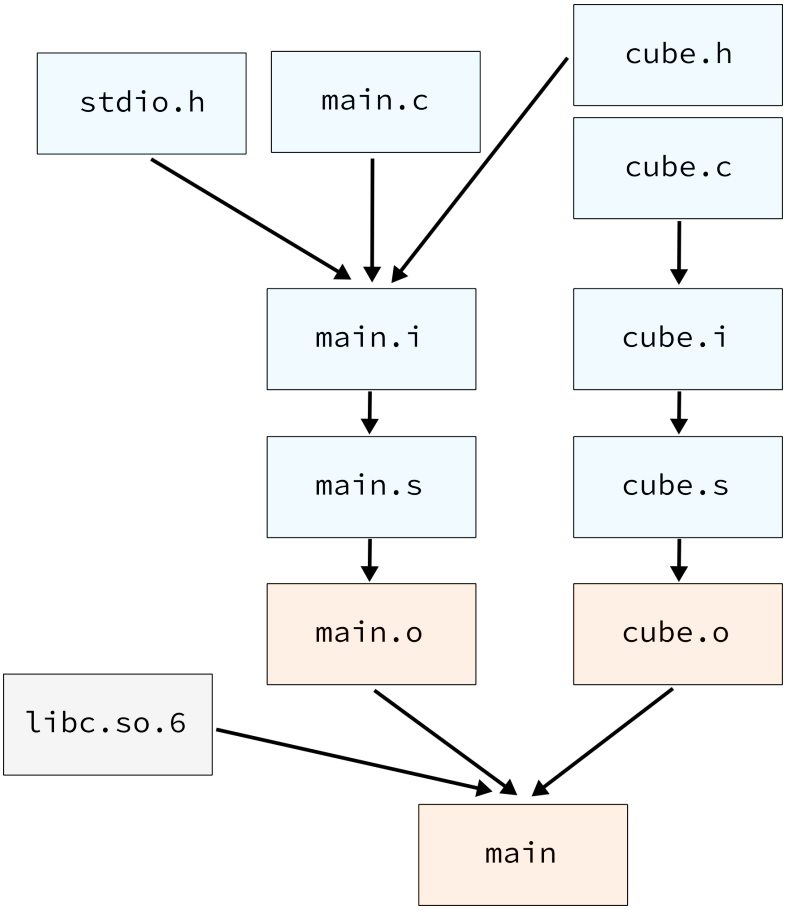
Но можно произвести компиляцию поэтапно, воспользовавшись соответствующими опциями компилятора. Например, мы можем сами скомпилировать отдельно файл `main.c` в объектный файл `main.o`, а файл `cube.c` в объектный файл `cube.o`, а после этого слинковать их вместе в итоговый исполняемый файл.

```
$ gcc -c main.c      # компилируем файл main.c отдельно и получаем файл main.o
$ gcc -c cube.c      # компилируем файл cube.c отдельно и получаем файл cube.o
$ gcc main.o cube.o  # линкуем файлы main.o и cube.o и получаем исполняемый файл
```

Замечание об использовании заголовочных файлов

Как правило, файлы с расширением `.h` предназначения для того, чтобы содержать в себе только объявления (функций, структур, глобальных переменных) и макросы. Но несмотря на то, иногда в таких файлах содержится сам код функций (как мы это делали в Части 2). Это допустимо, но нужно понимать, что храня код функций в заголовочных файлах, мы лишаемся преимуществ, которые даёт раздельная компиляция.

Схема процесса раздельной компиляции нашей программы:



Преимущества раздельной компиляции

Само по себе разбиение программы на несколько файлов умеет преемущества, такие как:

- **Лучшая структура программы**
  Программу, разделённую на несколько файлов, можно сделвть более структурированной. Код разбивается на части, каждая из которых отвечает за определённую функциональность.

- **Повторное использование кода**
  Код из одного проекта легче использовать в других проектах.

Однако, разбить код на части можно, используя только директиву `#include`, не прибегая к раздельной компиляции. Использование раздельной компиляции по сравнению с `#include` имеет следующие преимущества:

- **Ускорение перекомпиляции после изменений**
  Это самое главное преимущество раздельной компиляции. Чтобы его понять, представьте, что у вас есть большая программа, состоящая из сотен файлов. Предположим, что мы сделали маленькое изменение в одном из файлов и хотим заново скомпилировать программу. Если бы мы не использовали раздельную компиляцию (а просто бы подключали через `#include`), то нам бы пришлось перекомпилировать всю программу. Но, если мы используем раздельную компиляцию и сохранили объектные файлы со времени предыдущей компиляции, то нам будет достаточно перекомпилировать только файл, в котором были сделаны изменения. Этап линковки, всё же, придётся повторить для всей программы. Тем не менее, это всё равно будет намного быстрее компиляции всей программы.

- **Более точная настройка компиляции**
  Дальше в этом семинаре мы разберём, что компилировать программу можно разными способами, передавая компилятору разные опции. Использование раздельной компиляции позволяет использовать разные опции для разных файлов программы. Иногда это бывает полезно.

Ещё один пример программы, использующей раздельную компиляцию

Рассмотрим немного более сложную программу:

```
#include <stdio.h>
#include <math.h>

struct point
{
    float x;
    float y;
};
typedef struct point Point;

float distance(Point a, Point b)
{
    Point c = {a.x - b.x, a.y - b.y};
    return sqrt(c.x * c.x + c.y * c.y);
}

float triangle_area(Point a, Point b, Point c)
{
    float sa = distance(b, c);
    float sb = distance(c, a);
    float sc = distance(a, b);
    float p = (sa + sb + sc) / 2.0f;
    return sqrt(p * (p - sa) * (p - sb) * (p - sc));
}

int main()
{
    Point a = {0, 0};
    Point b = {10, 20};
    Point c = {20, 5};
    printf("%f\n", triangle_area(a, b, c));
}
```

Разбить такую программу на несколько файлов, для последующего применения раздельной компиляции, можно следующим образом. Обратите внимание, что объявление структуры `Point` тоже помещается в заголовочный файл. Далее, мы включаем файл `point.h` во все файлы, где используется структура `Point`.

main.c

```
#include <stdio.h>
#include "point.h"
#include "area.h"

int main()
{
    Point a = {0, 0};
    Point b = {10, 20};
    Point c = {20, 5};
    printf("%f\n", triangle_area(a, b, c));
}
```

point.h

```
#pragma once
struct point
{
    float x;
    float y;
};
typedef struct point Point;

float distance(Point, Point);
```

point.c

```
#include <math.h>
#include "point.h"

float distance(Point a, Point b)
{
    Point c = {a.x - b.x, a.y - b.y};
    return sqrt(c.x * c.x + c.y * c.y);
}
```

area.h

```
#pragma once
#include "point.h"
float triangle_area(Point, Point, Point);
```

area.c

```
#include <math.h>
#include "point.h"

float triangle_area(Point a, Point b, Point c)
{
    float sa = distance(b, c);
    float sb = distance(c, a);
    float sc = distance(a, b);
    float p = (sa + sb + sc) / 2.0f;
    return sqrt(p * (p - sa) * (p - sb) * (p - sc));
}
```

Часть 4: Опции компиляции

Опции компиляции – это параметры, которые передаются компилятору через командную строку для управления процессом компиляции. Они позволяют настраивать поведение компилятора, указывать пути к файлам, задавать уровни оптимизации, стандарты языка и многое другое.

Часть 5: Статические библиотеки. Утилита `ar`.

Часть 6: Преобразование кода на языке C в бинарный код

Часть 7: Формат файла ELF

Часть 8: Процесс линковки объектных файлов

Часть 9: Статические и внешние символы

Часть 10: Ключевое слово `inline`

Часть 11: Особенности компиляции и линковки на языке C++

Часть 12: Особенности компиляции и линковки статических библиотек в Windows